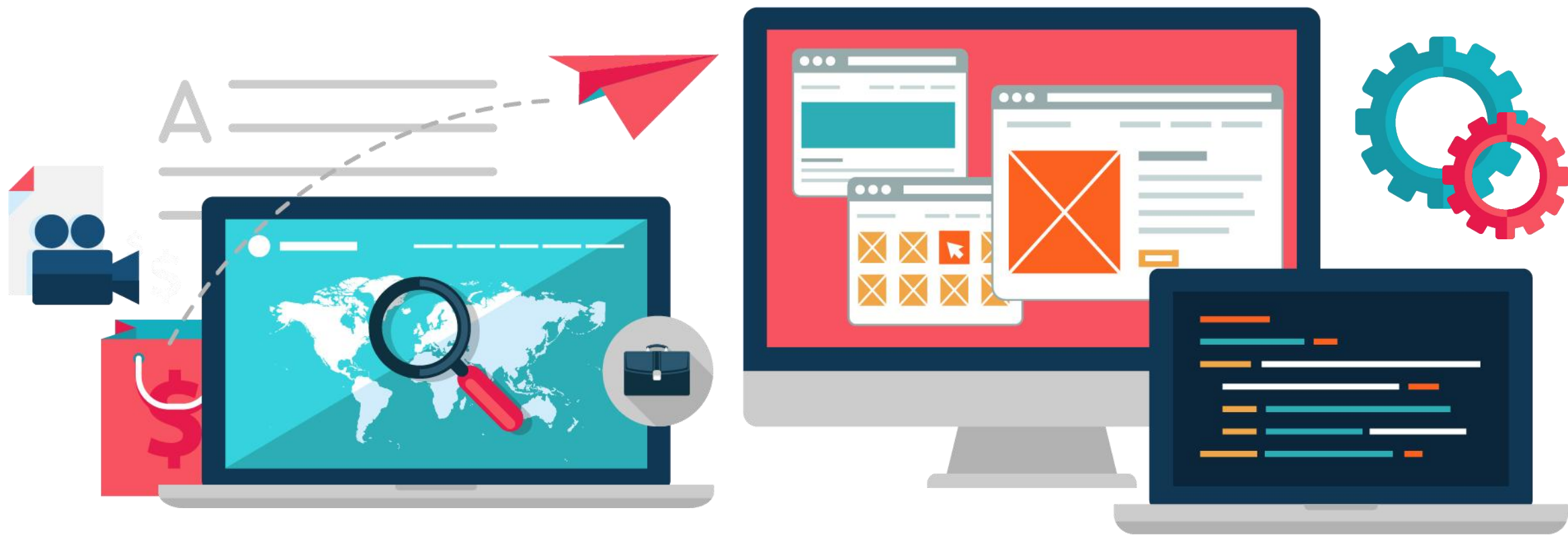


面向对象程序设计

Object Oriented Programming





多重继承



01

多重继承

02

多重继承中的命名冲突

03

菱形结构

04

解决方案-虚基类

05

其它解决方案





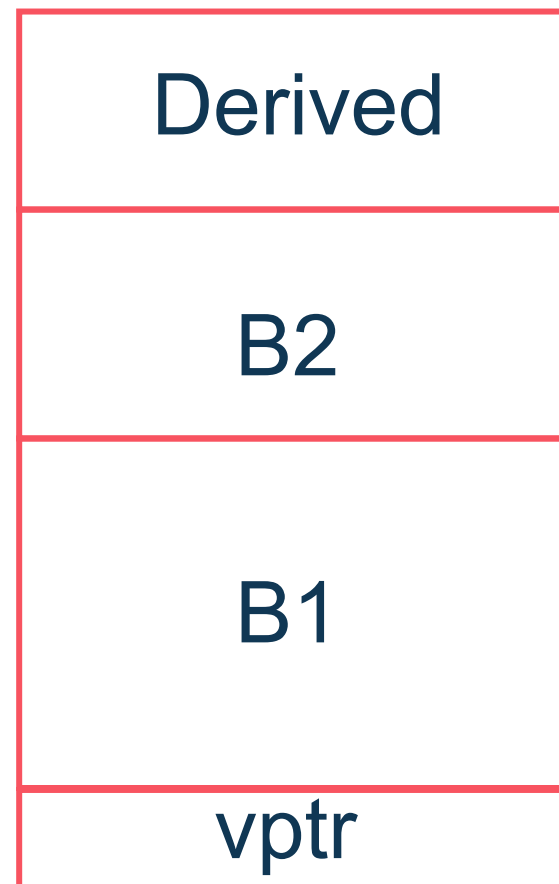
◆ 格式（例）：

```
class Derived: public B1,protected B2
{
    /* 内容 */
};
```

◆ 派生类的构造和析构的顺序

- ◆ 构造时，严格按继承顺序构造。
- ◆ 析构时，正好与构造顺序相反。

◆ 派生类对象的数据存放格式



```
class A {  
public:  
    A():num(1) {}  
    int f() { return 55; }  
protected:  
    int num;  
};
```

```
class B {  
public:  
    B():num(2) {}  
    int f() { return 88; }  
protected:  
    int num;  
};
```

```
class C: public A, public B {  
public:  
    void k() {  
        cout<< f() <<endl;           //模棱两可错误  
        cout<<num<<endl;           //模棱两可错误  
        cout<< B::f() <<endl;       //正确  
        cout<< A::num <<endl;       //正确  
    }  
};
```



使用using解决名字冲突

```
class A {  
public:  
    A():num(1) {}  
    int f() { return 55; }  
protected:  
    int num;  
};
```

```
class B {  
public:  
    B():num(2) {}  
    int f() { return 88; }  
protected:  
    int num;  
};
```

```
class C: public A,public B {  
public:  
    using A::f;  
    using B::num;  
    void k() {  
        cout<< f() <<endl; //A::f()  
        cout<<num<<endl; //B::num  
    }  
};
```



名字 冲突

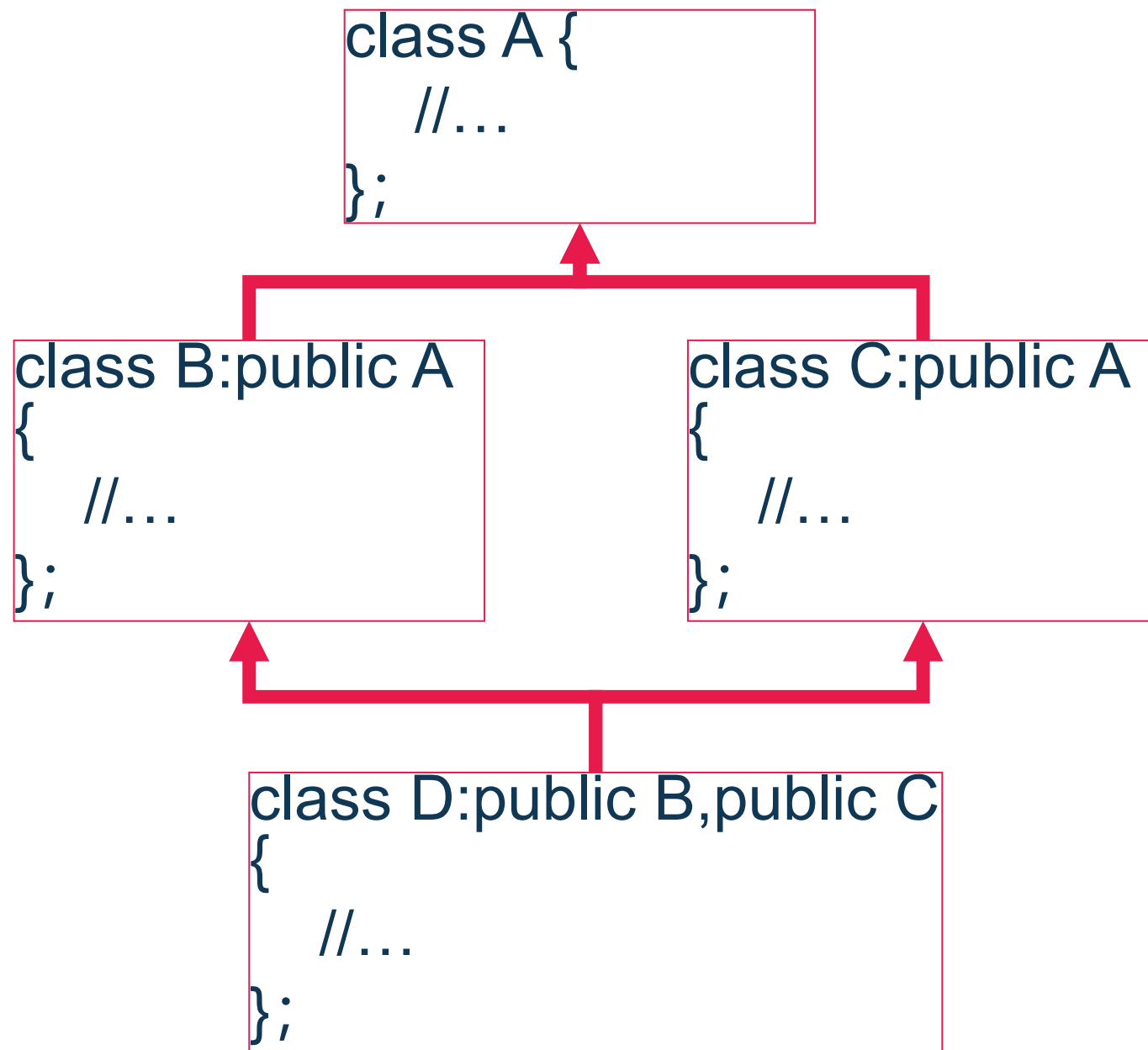
理想情况：
各函数、数据成员的名字，互不相同

其它情况：
使用using指明

真的解决了吗？



多重继承中的菱形结构(钻石结构)



名字不重复的菱形结构(例)

在该结构中，由于类A是派生类D两条继承路径上的一个公共基类，那么类D的对象将包含两个类A的子对象mA

```
int main( ) {  
    D d;  
    d.FA( );  
}
```

```
class A {  
    public:  
        void FA( );  
    private:  
        int mA;  
};
```

```
class B: public A {  
    public:  
        void FB( );  
    private:  
        int mB;  
};
```

```
class C: public A {  
    public:  
        void FC( );  
    private:  
        int mC;  
};
```

D中两个mA，同名但地址不同；但D中两个FA函数，同名且地址相同。因此解决实例变量的地址冲突(数据冗余)问题成为虚基类的主要目的

```
class D: public B, public C {  
    public:  
        void FD( );  
    private:  
        int mD;  
};
```



C++中虚基类的解决方案

```
int main( ) {  
    D d;  
    d.FA( );  
}
```

```
class A {  
    public:  
        void FA( );  
    private:  
        int mA;  
};
```

```
class B: virtual public A {  
    public:  
        void FB( );  
    private:  
        int mB;  
};
```

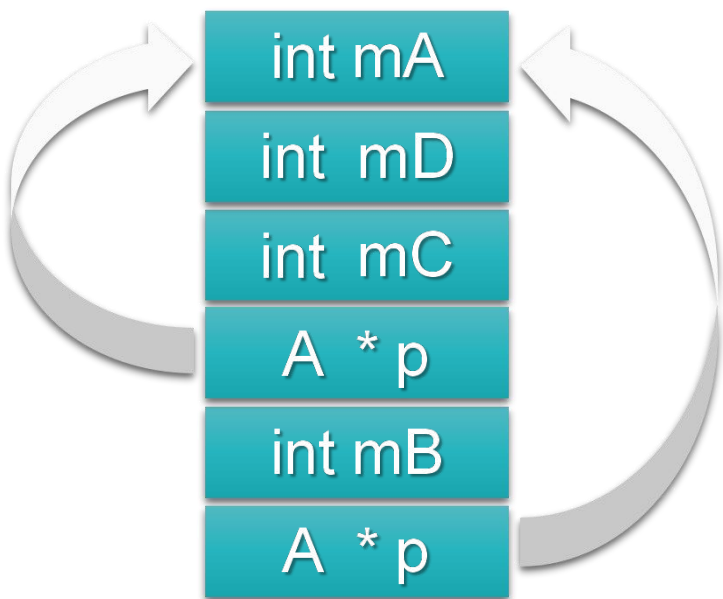
```
class C: virtual public A {  
    public:  
        void FC( );  
    private:  
        int mC;  
};
```

virtual是虚基类的关键字。虚基类的声明是在定义派生类时，写在派生类名的后面，共同基类即被声明为虚基类

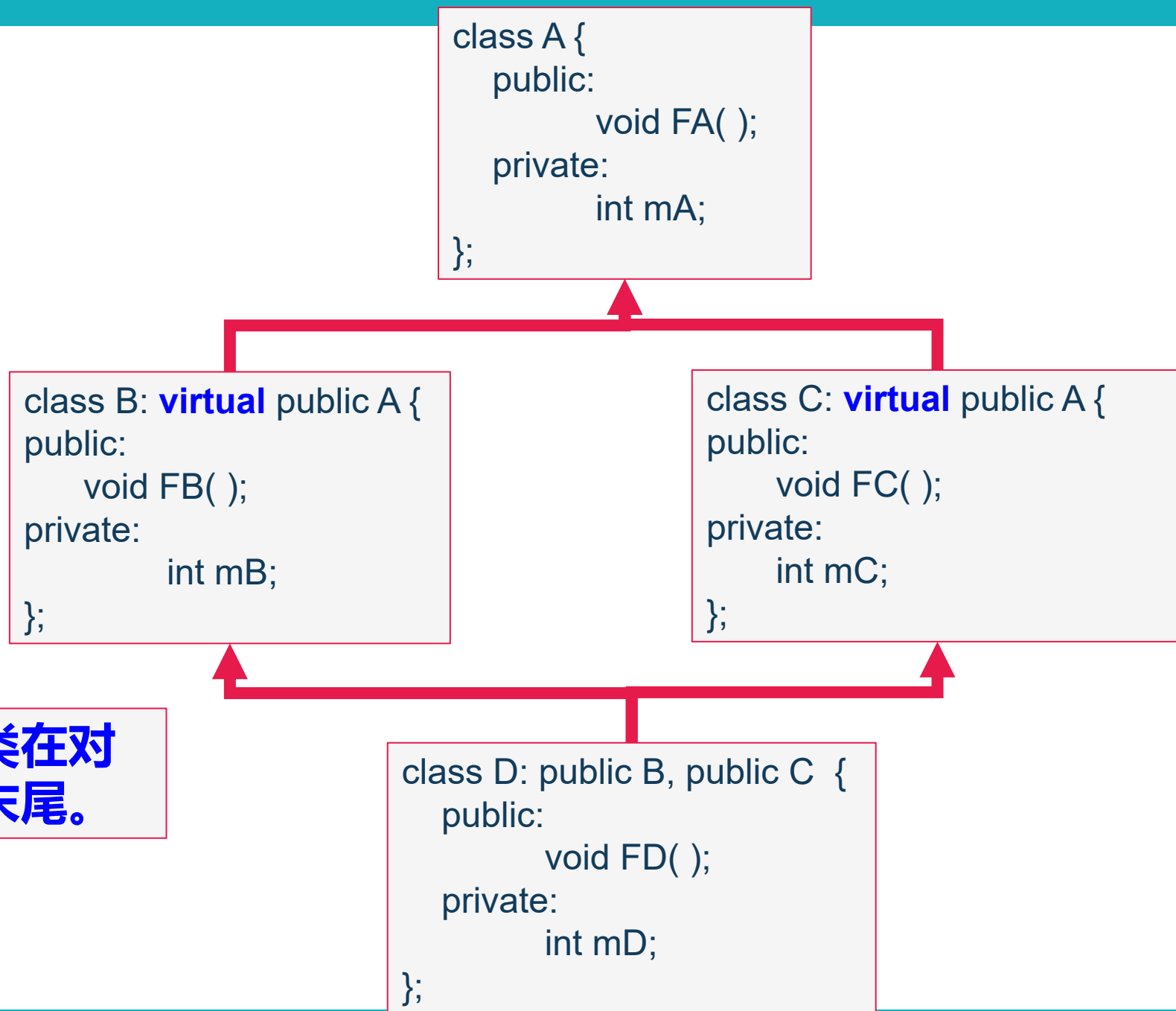
```
class D: public B, public C {  
    public:  
        void FD( );  
    private:  
        int mD;  
};
```



C++中虚基类的解决方案



虚基类对象的内存布局：虚基类在对象中仅存一份实例，位于布局末尾。





- ◆ 虚基类通过确保在继承链中虚基类的子对象仅存在一份实例，避免菱形继承导致的冗余。这通过以下机制实现：
- ◆ 最派生类负责初始化：虚基类的构造由最派生的类直接完成，而非中间基类。
- ◆ 间接访问成员：通过指针或偏移量访问虚基类成员，而非直接嵌入基类子对象。



- ◆ 派生类对象中只有一个虚基类子对象，为保证虚基类子对象只被初始化一次，虚基类构造函数必须只被调用一次
- ◆ 规定将在建立对象时所指定的类称为最派生类
- ◆ 虚基类子对象由最派生类的构造函数通过直接调用虚基类构造函数进行初始化
- ◆ 只要派生类有直接或间接的虚基类，那么就应该在派生类构造函数初始化列表显式调用虚基类构造函数。但是，只有最派生类的构造函数会真正调用虚基类的构造函数，其它派生类中对虚基类构造函数的调用将被忽略，以确保对虚基类对象只初始化一次
- ◆ 如果没有在最派生类的构造函数初始化列表显式调用虚基类构造函数，编译器会在该处自动调用虚基类的缺省构造函数对虚基类子对象初始化



```
class A {
public:
    A(){a=1;} A(int na):a(na){ }
    void f( ){ };
protected: int a;
};
class B1:virtual public A {
public:
    B1():A(){cout<<"B1:"<<a<<endl;}
};
class B2:virtual public A {
public:
    B2():A(){cout<<"B2:"<<a<<endl;}
};
```

```
class C:public B1,B2 {
public:
    C(int na):A(na),B1(),B2()
    {cout<<"C:"<<a<<endl;}
    void g( ){ f( ); }
};
int main( ) {
    C obj(2);
    obj.f( );
    return 0;
}
```



- ◆ 限定只能单继承
- ◆ 限定多个基类中，至多只能有一个基类有实例变量
 - ◆ 避免了多重继承下数据成员名字冲突的问题
 - ◆ 保留了多重继承的方便性

```
//有实例变量
class Person {
public:
    void study( );
    void work( );
    // ...
private:
    int salary;
};
```

```
//无实例变量
class Utility
{
public:
    static void f1( );
};
class MyException
{
};
```

```
//例：多重继承
class SpecialPerson
:public Person, public Utility, public
MyException
{
public:
    ...
};
```

◆ 常见的无实例变量的类

◆ 用类型做区分标志

◆ 工具类：类中只放置多个工具

◆ 接口类：指明其后裔类的公共行为，但其后裔类可实例化。接口类只含公有的纯虚实例方法，不包

◆ 接口继承：基类中至多只有一个接口

//用类型做区分标志（例）

```
void f() {  
    //抛出整型异常  
    if( ... )  
        throw 1;  
    else  
        throw 2;  
}
```

对比：

```
class ENormal { };  
class ESpecial { };
```

```
void f() {  
    //抛出自定义异常  
    if( ... )  
        throw ENormal();  
    else  
        throw ESpecial();  
}
```

通常接口类不能实例化，
只包含public行为，通常只包



- ◆ 常见的无实例变量的类
 - ◆ 用类型做区分标志
 - ◆ 工具类：类中只放置多个工具函数或类变量
 - ◆ 接口类：指明其后裔类的公共行为集(也称行为接口)，通常接口类不能实例化，但其后裔类可实例化。接口类无实例变量，一般只给出public行为，通常只包含公有的纯虚实例方法，不包含静态方法
- ◆ 接口继承：基类中至少

//工具类(例)

```
class PackUtility
{
public:
    static void PackBinary( );
    static void PackText( );
    static void PackPicture( );
    static int  getVer( );
private:
    static int  version;
};
```

类

◆ //例：普通父类

```
class Telephone {
public:
    ...
    void callTo( );
    void callBy( );
private:
    ....
};
```

◆ //例：接口类

```
class IComputer {
public:
    static const int xx = 1;
public:
    virtual ~IComputer( );
    virtual void runApp( );
    virtual void showVideo( );
};
```

◆ //例：接口类

```
class IPaper {
public:
    virtual ~IPaper( );
    virtual void writeText( );
};
```

实例化，
一般只给出public行为，通常只包

勾为接口类

◆ //例：接口继承

```
class Mobile:public Telephone,public IComputer,public IPaper
{
public:
    void callTo( );
    void callBy( );
    virtual void runApp( );
    virtual void showVideo( );
    virtual void writeText( );
};
```



本章结束